Vectors Part 3

Instructor: Dr. Khaled Mohamad

In [1]:
```
#Import numpy and Matplotlib modules
import numpy as np
import matplotlib.pyplot as plt

#ignore warning message
import warnings
warnings.filterwarnings('ignore')
```

Vector length

In [2]:
```
# a vector
a = np.array([ 1, 2, 3, 4, 5, 6 ])

# methods 1, just like with the regular dot product
al1 = np.sqrt( sum( np.multiply(a,a)) )

# method 2: take the norm
al2 = np.linalg.norm(a)

print(f"Method 1: {al1}\n")
print(f"Method 2: {al2}\n")
```

```
Method 1: 9.539392014169456

Method 2: 9.539392014169456
```

The dot product (Vector-Vector multiplication)

In [3]:
```
## many ways to compute the dot product
a = np.array([ 1, 2, 3, 4, 5, 6 ])
b = np.array([ 0, -4, 3, -3, 6, 5 ])

#Option 1
# Use a for loop over elements
ab1 = 0  # initialize
for i in range(len(a)):
    # multiply corresponding element and sum
    ab1 = ab1 + a[i]*b[i]


# Option 2
ab2 = np.dot( a,b )


# Option 3
ab3 = sum( np.multiply(a,b) )


# Option 4
ab4 = np.matmul( a,b )

print(ab1,ab2,ab3,ab4)
```

```
49 49 49 49
```

Dot product properties:

Distributive

In [4]:
```
# create 3 random vectors
n = 10
a = np.random.randn(n)
b = np.random.randn(n)
c = np.random.randn(n)

# Results
r1 = np.dot(a,b) + np.dot(a,c)
r2 = np.dot( a , (b+c) )
```

```
# Result with comparison
print([ r1,r2 ])
```

```
[np.float64(6.270755599955152), np.float64(6.270755599955152)]
```

Associative

Special cases where associative property defined

- 1. One vector is zero vector
- 2. a=b=c

In [5]:
```
# Results
r_1 = np.dot( a , np.dot(b,c) )
r_2 = np.dot( np.dot(b,c) , a )

# Result with comparison
print(r_1)
print(r_2)
```

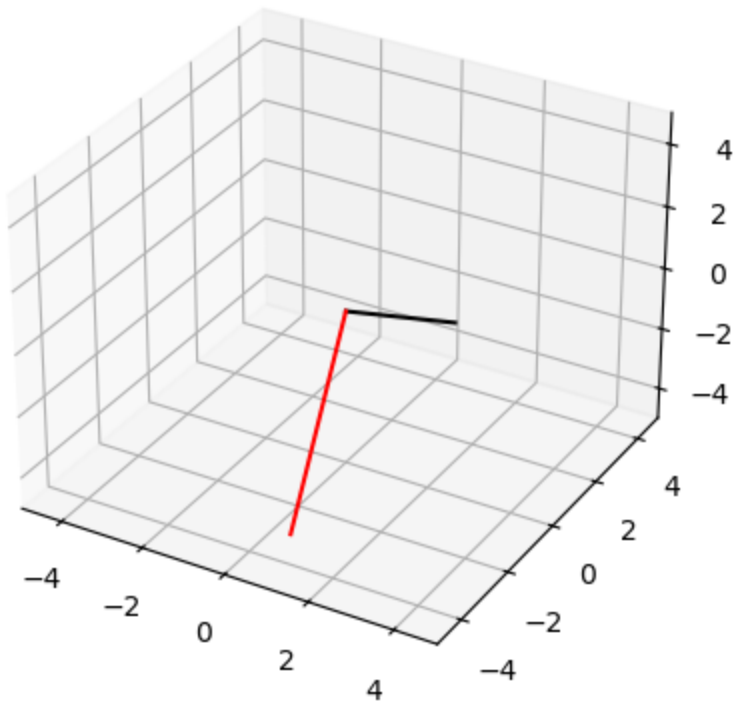
```
[-1.62500826  0.637267    0.21777935  0.02334235 -0.20363474 -0.30395517
  2.08102697 -0.2718324  -0.523076    0.52271955]
[-1.62500826  0.637267    0.21777935  0.02334235 -0.20363474 -0.30395517
  2.08102697 -0.2718324  -0.523076    0.52271955]
```

The Geometric perspective of the dot product

In [6]:
```
# define two vectors
a = np.array([ 1,  3, -2 ])
b = np.array([ 1, -4, -4 ])

# compute the angle (radians) between two vectors
ang = np.arccos( np.dot(a,b) / (np.linalg.norm(a)*np.linalg.norm(b)) )
ang_deg = np.round((np.degrees(ang)),2)

# draw 3D
fig = plt.figure()
ax = plt.axes(projection = '3d')
```

```
ax.plot([0, a[0]],[0, a[1]],[0, a[2]],'k')
ax.plot([0, b[0]],[0, b[1]],[0, b[2]],'r')

plt.axis((-5, 5, -5, 5, -5, 5))
plt.title('The angle betweenvectors a and b: %s deg.' %ang_deg)
plt.show()
```

The angle betweenvectors a and b: 98.02 deg.



Check if the algebraic and geometric formulas of dot product are the same or not

In [7]:
```
# algebraic formula
ab_a = np.dot( a,b )

# geometric formula
ab_g = np.linalg.norm(a)*np.linalg.norm(b)*np.cos(ang)

# Result and compare
```

```
print(ab_a)
print(ab_g)
```

```
-3
-2.9999999999999982
```

Hadamard multiplication

In [8]:
```
a = [ 1, 3, 5 ]
b = [ 3, 4, 2 ]

ab_h = np.multiply(a,b)
print(ab_h)
```

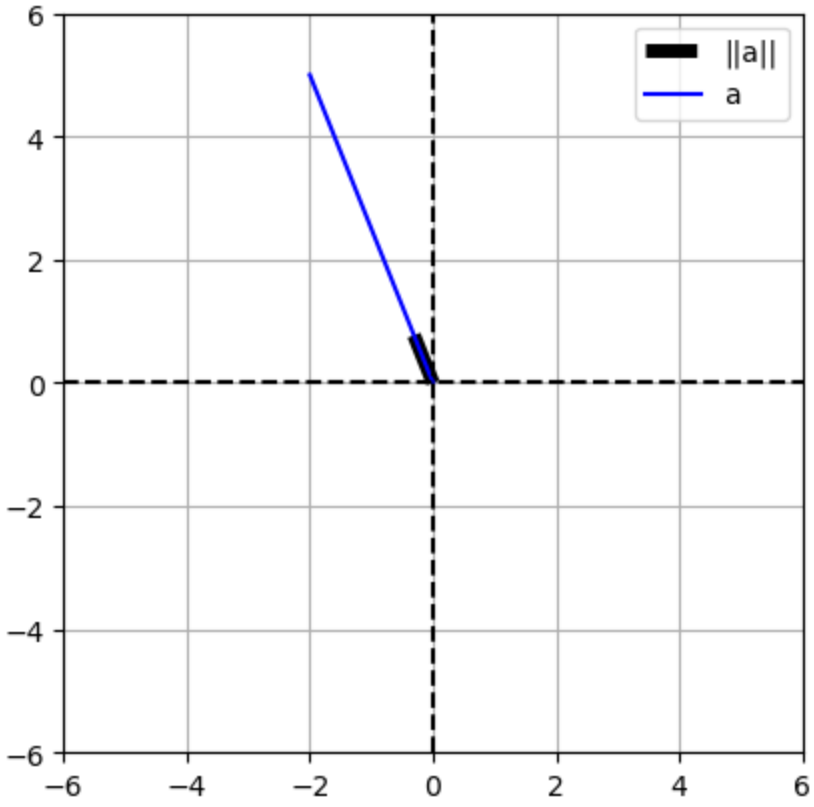
```
[ 3 12 10]
```

Outer product

In [9]:
```
#outer product
op = np.outer(a,b)
print(op)
```

```
[[ 3  4  2]
 [ 9 12  6]
 [15 20 10]]
```

Unit vector

In [10]:
```
# identify a vector
a = np.array([ -2, 5 ])

# 1/length
sca = 1/np.linalg.norm(a)

a_sca  = a*sca

# plot them
plt.plot([0, a_sca[0]],[0, a_sca[1]],'k--',label='||a||',linewidth=5)
plt.plot([0, a[0]],[0, a[1]],'b',label='a')
```

```
#optional to add dashed axies

plt.plot([-6,6], [0,0], "k--")
plt.plot([0,0], [-6,6],"k--")

plt.axis('square')
plt.axis(( -6, 6, -6, 6 ))
plt.grid()
plt.legend()
plt.show()
```